# Systems and Devices 3 (SYS3)

## Lecture Review – Week 2

Dr Pengcheng Liu

# Welcome

**Check-In Code:**

# 639079

Log into [checkin.york.ac.uk](checkin.york.ac.uk), find the session and enter the code.

Valid until Thursday, February 19, 2026 11:00 AM

# Agenda and Objectives

**Module Overview and Updates**

- Provide important announcements and updates about the module.
- Highlight key themes and objectives from the week's lecture.
- Introduce the guest lecturer (if applicable) and their expertise.

**Pre-Submitted Questions Discussion**

- Discuss questions submitted in advance through Padlet.
- Clarify any complex or challenging concepts from the lecture material.

**Guest Lecture and Insights (if applicable)**

- Provide an industry or research-based perspective related to the topic.
- Share real-world applications, case studies, or expert insights.
- Allow students to engage with the guest lecturer through Q&A.

# Agenda and Objectives

**Exercises/Quizzes Discussion**

- Go through exercises, quizzes, or problem sets related to the topic. (See related timeline on VLE)
- Offer explanations and insights to reinforce learning.

**In-the-Classroom Discussion Forum**

- Facilitate discussions on key themes, real-world applications, and guest lecture insights.
- Encourage students to share their opinions, experiences, and critical thoughts.

**Closing Remarks and Announcements**

- Summarise key takeaways from the session, including guest lecture insights.
- Provide guidance on upcoming topics, assessments, or deadlines. Provide announcements for upcoming events or relevant opportunities.

# Week 2 Module
# Overview and Updates

# Module Overview and Updates

- **Week 2**:

**Lecture:** This week, our discussion was focused on some basics of computer organization and we discussed some principles of computer design.

**Lab:** Orientation Practical. Performed activities to familiarise with the PIPELINE TOOLSET used in these lab sessions, and to understand the basic working principles of using the software for experiments in week 2-11 of this semester.

**Exercises/Quizzes:** Exercise 1 was released on 13/02/2026 (Week 1 Friday), which covers Computer Architecture Introduction, Computer Design Principles, Pipeline Basics, Structural and Data Hazards. Please try to complete it by 06/03/2026 (Week 4 Friday). We will use the Lecture Review session on 12/03/2026 (Week 5 Thursday) for discussions if there are any questions on it.

# Module Overview and Updates

- **Next Week (Week 3)**:

**Lecture:** Next week, our discussion will be focused on some principles of the RISC Pipeline. Also, we will be having our first Tutorial Lecture on Performance.

**Lab:** In Weeks 3 and 4 we will be exploring the idea of structural hazards in a CPU pipeline, and possible solutions. You will start with some basic ideas, then develop an algorithm to solve structural hazards in the pipeline. In Week4 an addittional help sheet will be released to assist those who have not been able to devise a suitable idea to implement. Solutions to algorithm problems will be released at a later date.

**Exercises/Quizzes:** Quiz 1 will be release on 20/02/2026 (Week 2 Friday). This is a formative quiz based on week 1, week 2 and week 3 lectures, responses will be automarked. You can try as many times as you like till you get a perfect score. Quiz 1 will be discussed in the Lecture Review session on 12/03/2026 (Week 5 Thursday).

# Lecture Review Plan

Week 1: Introduction and Briefing

Week 2: Regular Lecture Review

Week 3: Guest Lecture 1:

- RISC-V: A Brief History and Roadmap to 2036

Week 4: Guest Lecture 2:

- Advanced Computer Systems for AI & Computer Vision

Week 5: Quiz 1 & Exercise 1 Discussion

Week 6: Regular Lecture Review

Week 7: Quiz 2 Discussion

Week 8: Exercise 2 Discussion

Week 9: Quiz 3 Discussion

Week 10: Exercise 3 Discussion

**Week 3 Lecture Review**

**Guest Lecture 1**

**Speaker: Dr Wei Wu**

**Title:** RISC-V: A Brief History and Roadmap to 2036

**Abstract:** As a young and open ISA standard, the RISC-V has the potential to capture a quarter of the global processor market share within its 20-year lifespan. Some bold predictions even suggest that RISC-V will become the most dominative ISA in the worldwide IT industry by 2036, serving as a cornerstone of the global open-source ecosystem due to its open and free nature.

In this lecture, the speaker will begin with the historical context of RISC-V's emergence, explaining the historical opportunities and inevitability of its creation as an open and free ISA. Furthermore, the speaker will showcase the remarkable achievements RISC-V has made in the software and open-source chip ecosystems, particularly the policy support from EU, China, Brazil, and India.

## Week 3 Lecture Review

## Guest Lecture 1

## Speaker: Dr Wei Wu

**Title:** RISC-V: A Brief History and Roadmap to 2036

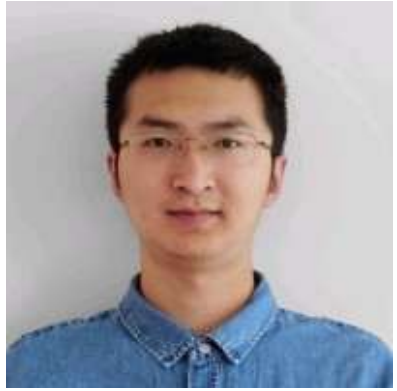
**Biography:** Mr. Wei Wu is the founder and director of Project Jiachen, whose mission is to make RISC-V the most dominant instruction set architecture for all IT scenarios before 2036, achieving commercial prosperity. Currently, more than 50 companies, research institutions, and developers worldwide have joined the project, covering almost all major RISC-V IP companies. To achieve this vision, he founded PLCT Lab in 2019 and serves as its project director. Improving the RISC-V software ecosystem has been set as one of PLCT lab's most important strategic objectives. His team maintains a series of key open-source infrastructures, including OpenJDK RISC-V backend, V8 RISC-V backend, Spidermonkey RISC-V JIT backend, LuaJIT RISC-V downstream implementation, as well as GNU Toolchain and Clang/LLVM RISC-V backend. The TARSIER project, which he launched at the end of 2021, has taken a significant step forward in RISC-V desktop computing and Linux distribution support. In October 2025, he joined KUBUDS, a company he co-founded in 2023, to help RISC-V companies achieve greater success in the commercial industrial software ecosystem.

Wei has served as a director of the LLVM Foundation Board of Directors (BoD) since 2022. He is very passionate about educating compiler techniques and RISC-V to engineers and amateurs. He often gives talks and tutorials among amateurs and college students in local meetups. He also collaborates with college teachers, providing RISC-V software lectures and experiment courses.

**Week 4 Lecture Review**

**Guest Lecture 2**

**Speaker: Dr Yan Wen**



**Title:** Advanced Computer Systems for AI and Computer Vision: From Parallel Architectures to Cloud Deployments

**Abstract:** This lecture shows how modern computing hardware helps build and run large-scale computer vision and AI applications. We will start by **comparing CPUs and GPUs**, and explain why doing many operations at once (parallelism) is key for tasks like **matrix multiplication and convolution** in deep learning. Next, we will explore parallel programming by demonstrating a computer vision algorithm on both the CPU and GPU using **PyTorch**, highlighting performance comparisons. Then, we will look at how **AI runs on the cloud at large scale**, using tools like **Kubernetes**, container orchestration, and serverless computing to manage resources. We will also discuss **edge and embedded devices**, explaining how small systems-on-a-chip balance performance and power in tight, real-time conditions. Finally, we will see a case study of a complete **web application** powered by AI and cloud computing. From this lecture, students will understand how to connect these hardware ideas with real-world AI system design, whether running everything on-site or in the cloud.

# Pre-Submitted Questions on Padlet

# Padlet Questions

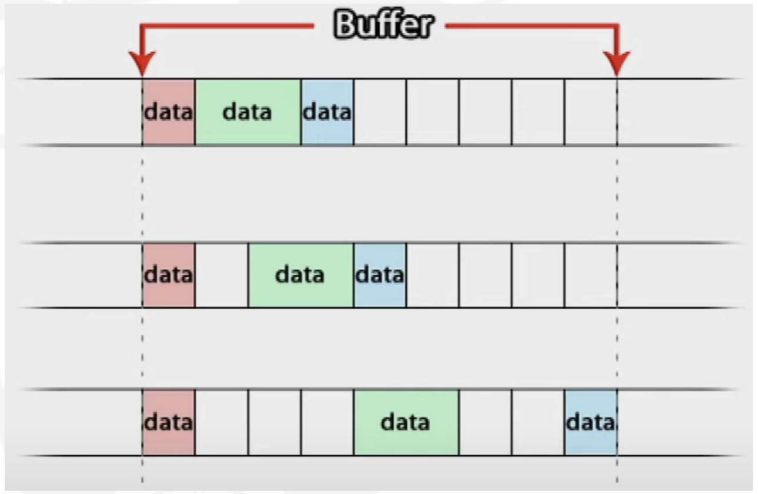
## S2 Week 2 L3 - Basics 1: Data Alignment

*The concept of aligning data makes sense - why do we do this? It seems inefficient from a memory management standpoint.*

Data alignment is used because it makes **memory access faster, simpler, and more efficient for the CPU**, even though it may appear to waste some memory space.

Processors are designed to read memory in fixed-size chunks called **words** (e.g., 4 bytes, 8 bytes). When data is aligned to its natural boundary:

- A 4-byte integer aligned to a 4-byte boundary can be read in **one memory access**
- If it is misaligned (e.g., starts at address 3 instead of 4), the CPU may need **two memory accesses** to retrieve it.




Aligned to 1 byte

Aligned to 2 bytes

Aligned to 4 bytes

# Padlet Questions

## S2 Week 2 L3 - Basics 1: Data Alignment

*The concept of aligning data makes sense - why do we do this? It seems inefficient from a memory management standpoint.*

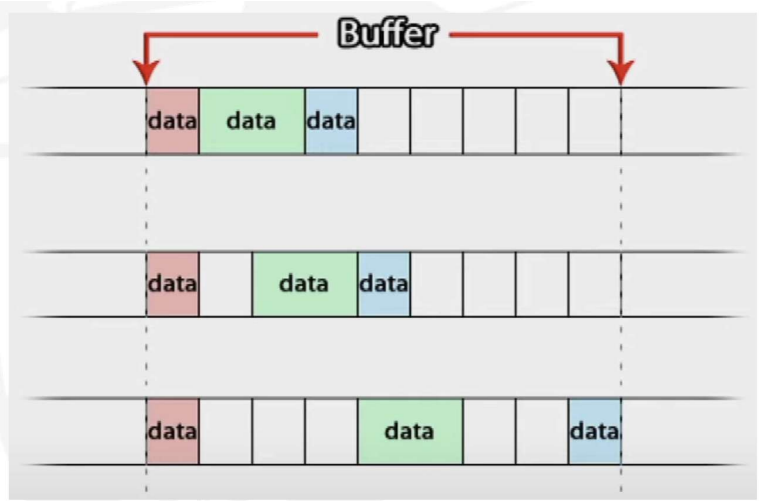
Aligned data allows the CPU and memory system to operate with **simpler and more efficient hardware design**:

- Memory buses are designed to fetch aligned chunks
- No extra logic needed to split and recombine data
- Reduces CPU complexity and power consumption

Misaligned access requires additional hardware handling, which is slower and more complex.

Aligned accesses require fewer clock cycles; avoid extra internal operations; improve overall instruction execution speed.

Misaligned accesses may cause multiple memory reads; extra CPU cycles; pipeline stalls.




Aligned to 1 byte

Aligned to 2 bytes

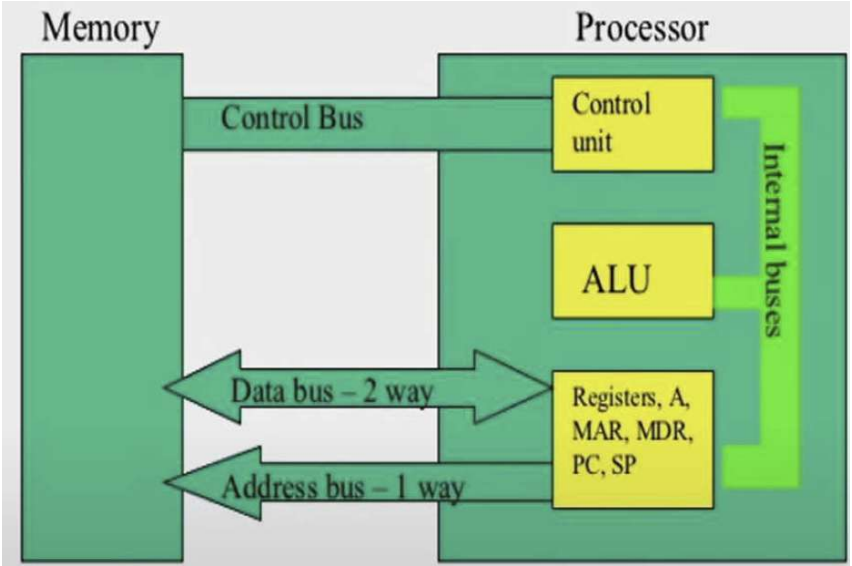
Aligned to 4 bytes

# Padlet Questions

## S2 Week 2 L3 - Basics 1: Control Bus Direction

*In the architecture diagram, the Data bus and Address bus have arrow heads, but the Control Bus does not. Is there something different about the Control bus that we should infer from this?*

The **data bus** typically has **arrowheads on both ends**, indicating that data can flow in both directions. It allows the processor to read from and write to memory or peripherals.
The **address bus** usually has a **single arrow** pointing from the processor to memory, indicating that the processor sends address information to memory but does not receive addresses back.



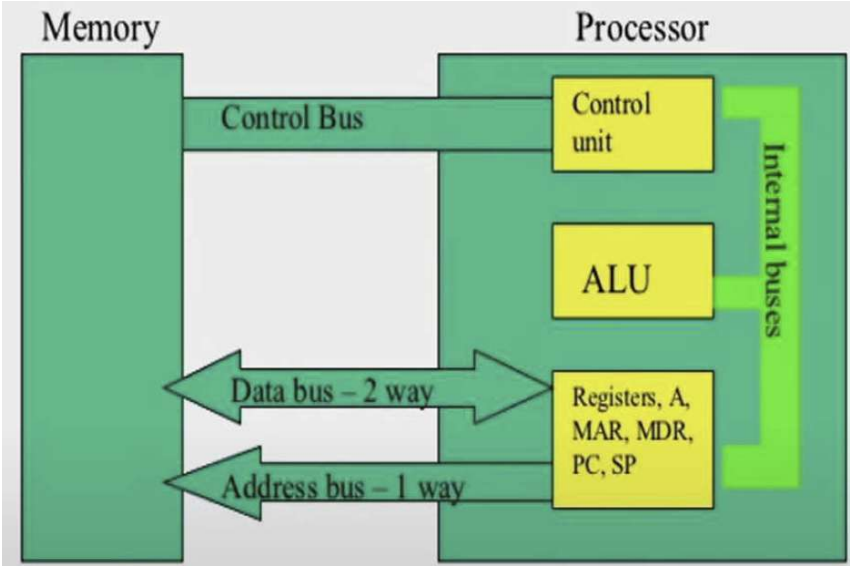
# Padlet Questions

## S2 Week 2 L3 - Basics 1: Control Bus Direction

*In the architecture diagram, the Data bus and Address bus have arrow heads, but the Control Bus does not. Is there something different about the Control bus that we should infer from this?*

The **control bus** has **varied directions** and is **context-dependent**. The absence of arrowheads on the control bus indicates that control signals flow in multiple directions, depending on the specific control line. Some signals (like memory read/write) go from the processor to memory, while others (like interrupt signals) may come from peripherals to the processor.

Because control signals are not necessarily constrained to a single direction, the diagram may omit arrowheads to reflect this flexibility.



# Padlet Questions

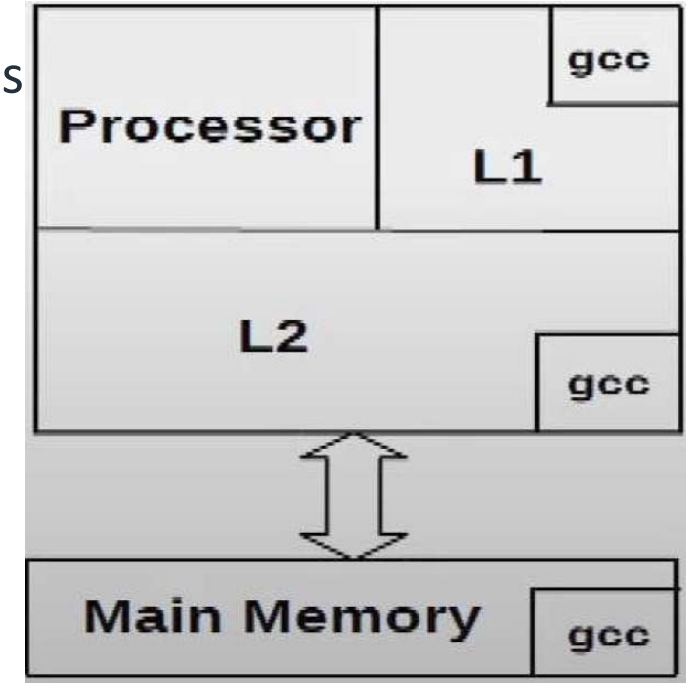
**S2 Week 2 L3 - Basics 1: Processor Caches**

*Are all programs brought into the L2 and L1 caches when they are run? Or just some that are frequently accessed?*

Not all programs are brought into the L1 and L2 caches when they are run. Instead, only the most frequently accessed and recently needed parts of a program (such as instructions and data) are loaded into the caches.

This process is managed dynamically by the CPU's cache management system.

We will cover this in the caches lectures later.



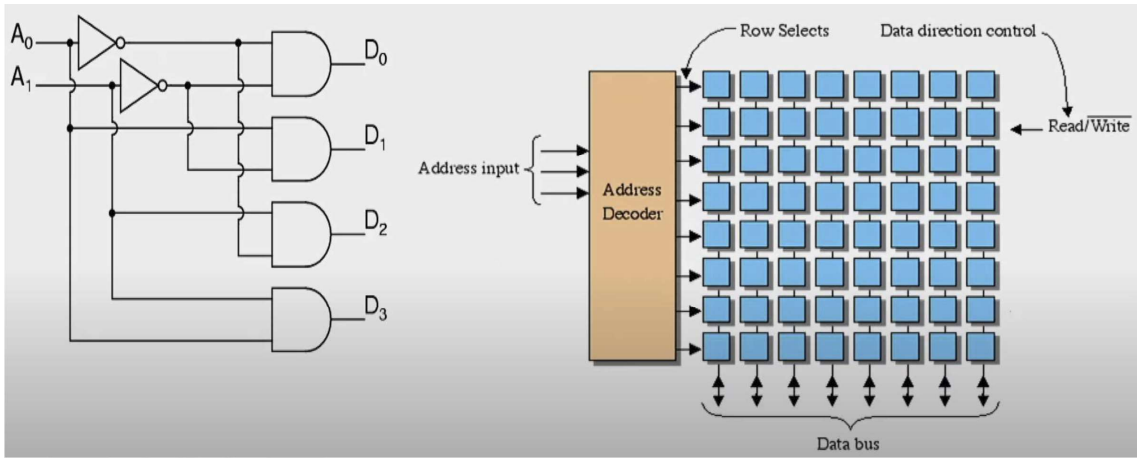
# Padlet Questions

## S2 Week 2 L3 - Basics 1: MAR Decoder

*Please can you explain what you said about the Decoder having 2^n outputs? What are these outputs?*

A decoder is a combinational circuit that takes an n-bit address input and produces 2^$n$ unique output lines. Each output corresponds to a unique memory location or memory block.

Each of the 2^$n$ outputs represents a specific memory location or memory chip enable signal:
If using a single memory chip, each output corresponds to a unique memory address. If using multiple memory chips, the decoder may select which chip should respond to a given address.



# Padlet Questions

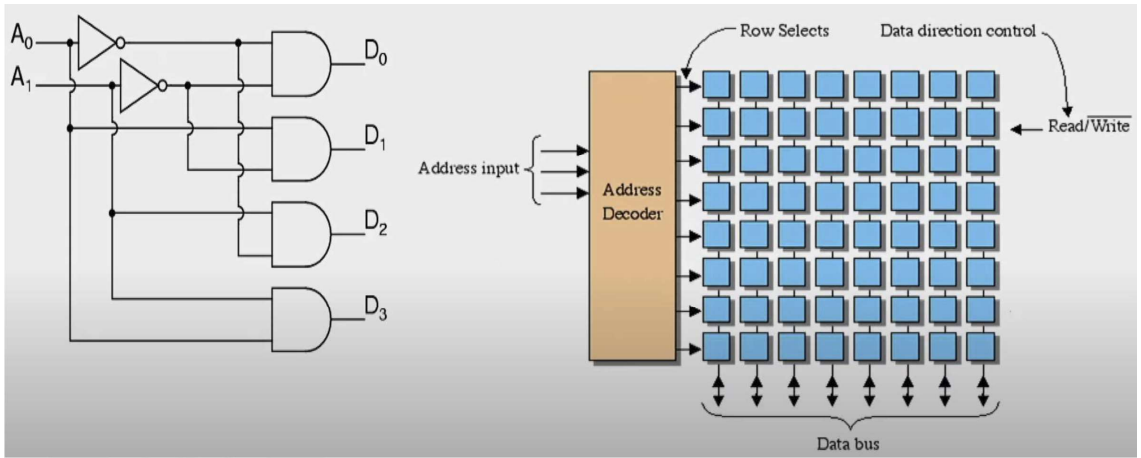
## S2 Week 2 L3 - Basics 1: MAR Decoder

Please can you explain what you said about the Decoder having 2^n outputs? What are these outputs?

**How it Works:**

Suppose the memory address bus has **n bits**.

This means it can represent **2^n unique address values** (because an n-bit binary number can take values from 0 to 2^n - 1).
The **decoder uses these n bits** to activate one of **2^n output lines**.
Each output line corresponds to a **unique memory location** or **memory chip selection**.



# Padlet Questions

## S2 Week 2 L3 - Basics 1: MAR Decoder

Please can you explain what you said about the Decoder having 2^n outputs? What are these outputs?

**Example:**

If the address bus has **2 bits** (n = 2):
It can represent 2^2 = 4 unique addresses (00, 01, 10, 11 in binary).
A **2-to-4 decoder** takes these **2 input bits** and activates one of **4 output lines**.
Each output line represents a **memory location or block** that is being accessed.

| Input (Address Bits) | Output Lines (One Active) |
|---|---|
| 00 | 0001 (Activates Line 1) |
| 01 | 0010 (Activates Line 2) |
| 10 | 0100 (Activates Line 3) |
| 11 | 1000 (Activates Line 4) |

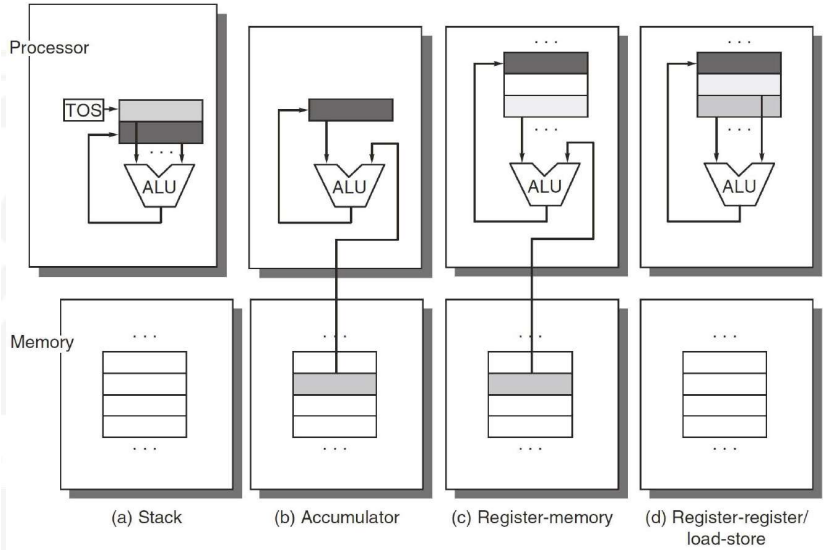
# Padlet Questions

## S2 Week 2 L3 - Basics 2: Stack Architecture & Calculations

*Am I understanding this correctly for Stack architecture?*

*`Add` pops the top two values in the stack into the ALU (pop meaning that they are no longer in the stack), then those values are summed and pushed back into the stack? (So essentially, `Add` replaces the top 2 values in the stack with one value that is the sum of them?)*

Add removes the two topmost values from the stack and produces a single result back onto the stack. The stack shrinks by one element after the operation. So Add replaces the top two values in the stack with their sum.




| Stack | Accumulator | Register (register-memory) | Register (load-store) |
|---|---|---|---|
| Push A | Load A | Load R1,A | Load R1,A |
| Push B | Add B | Add R3,R1,B | Load R2,B |
| Add | Store C | Store R3,C | Add R3,R1,R2 |
| Pop C | | | Store R3,C |

The code sequence for C = A + B for four classes of instruction sets.

# Padlet Questions

**S2 Week 2 L4 - Computer design 2: Slide 2: Transactions / Unit Time**

When you're talking about measuring the number of transactions a server can perform, would that usually be from the perspective of a second machine (i.e. including network too) or using some sort of local benchmarking tool?

It depends on the context of the benchmarking. Typically, measuring the number of transactions a server can perform (**transactions per unit time**) can be done in two ways: 1. Using **local benchmarking tools**. This approach tests the server's raw processing capability without network overhead. It typically uses a tool running on the server itself (or a directly connected system) to simulate and measure transactions.
2. **Client-server perspective** (including network). This approach measures **end-to-end** performance, including network latency and communication overhead. It involves a second machine acting as a client, sending requests to the server and measuring the number of completed transactions over time. This method is useful for real-world performance evaluations, especially for web services, APIs, and distributed systems.

# Padlet Questions

**S2 Week 2 L4 - Computer design 1: Typical performance metrics**

What's a good definition for speedup? I understand what it means but formally how would you describe it?

Speedup is the execution time of the original system (e.g., a single-core processor or an unoptimised program) over the execution time of the improved system (e.g., a multi-core processor, optimised code, or hardware upgrade).

$$\text{Speedup} = \frac{\text{Execution Time of the Original (Baseline) System}}{\text{Execution Time of the Improved System}}$$

Speedup = 1 → No performance improvement.
Speedup > 1 → The improved system is faster.
Speedup < 1 → The "improved" system is actually worse.
Speedup = N → The new system is N times faster than the old one.

# Padlet Questions

**S2 Week 2 L4 - Computer design 1: Typical performance metrics**

Whats a good definition for speedup? I understand what it means but formally how would you describe it?

The Amdahl's Law we discussed in the lecture is when improving only a portion of the execution, speedup is limited by the fraction of time that is actually improved.

$$S = \frac{1}{(1-P) + \frac{P}{S_{\text{enhanced}}}}$$

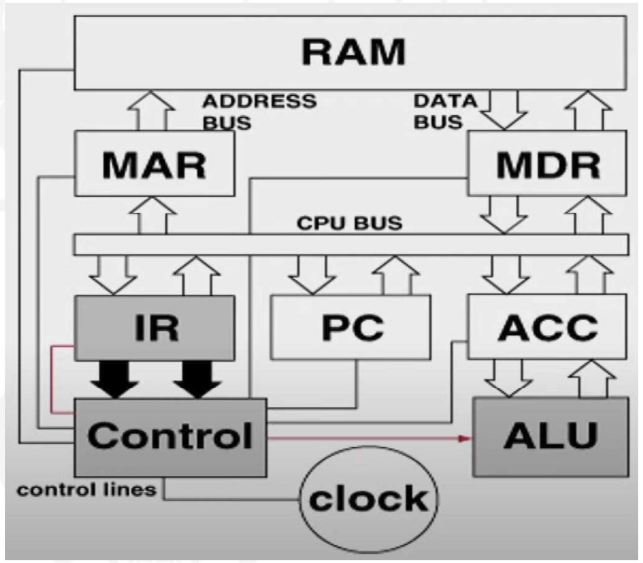
where P is the fraction of execution time affected by the improvement, and S_enhanced is the speedup of that portion.

# Padlet Questions

**L3 Basics Execute Stage:** *Can anything (and if so, what) can be done in the execute section other than arithmetic and logic operations?*

This will be covered in Lecture 5 where we discuss more details about the pipeline and pipeline stages. In a pipeline, Execute stage primarily handles arithmetic and logic operations, there are other tasks that can be performed. For example, for load/store instructions or other operations that involve **memory access**, the calculation of memory addresses occurs in the Execute stage.

**Execute the instruction**

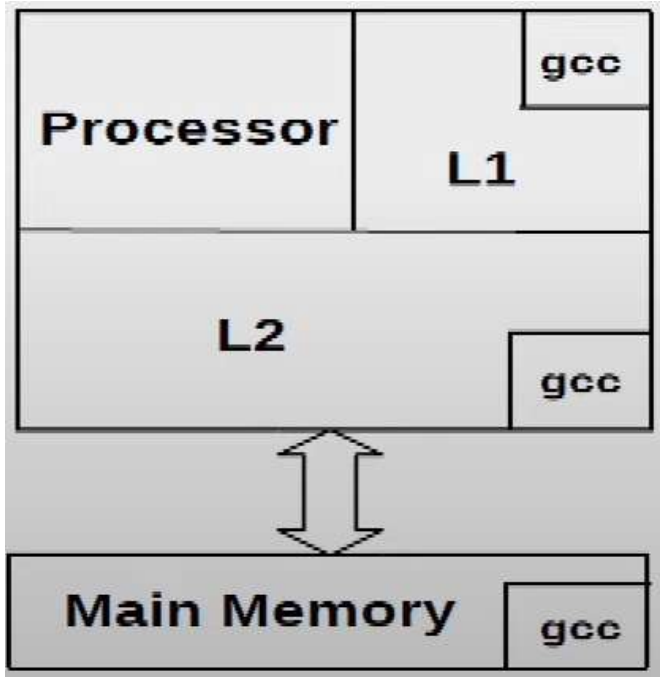



5. Control unit sends signals to appropriate devices to cause execution of the instruction.

# Padlet Questions

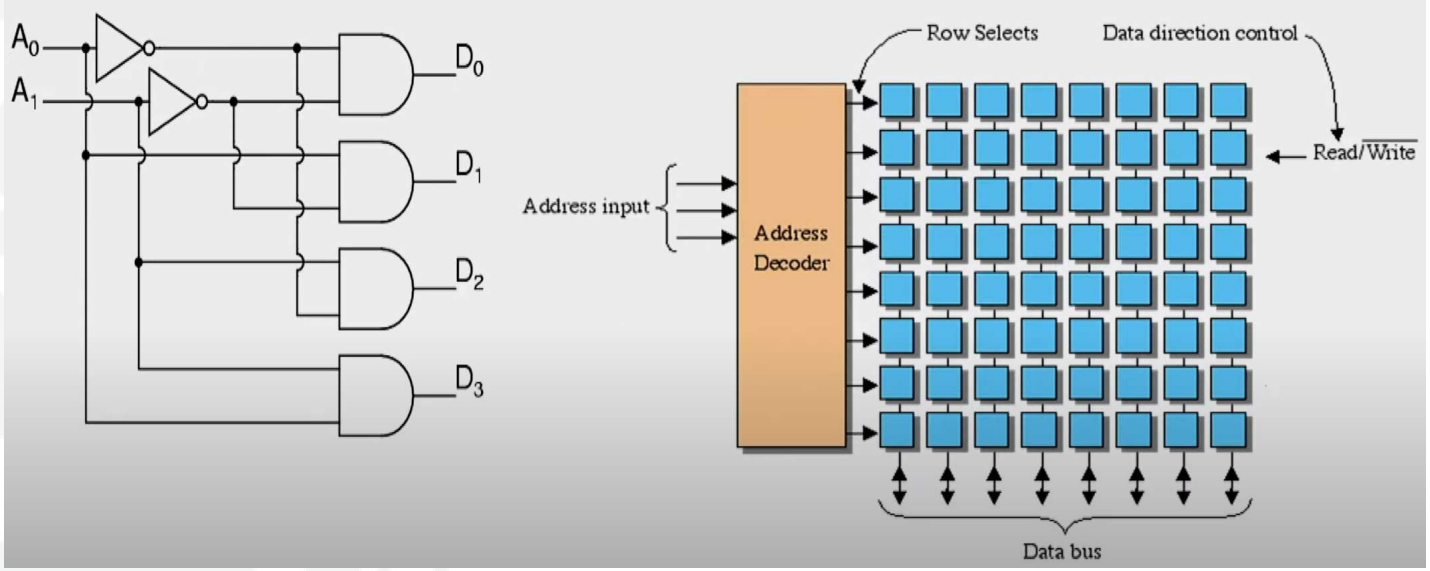
**L3 Basics Slide 4:** Can you further explain the gcc example on this slide?

This example mentions compilers and caches, which we will discuss in detail in the following lectures.
Generally, when a program runs, the CPU accesses memory locations to load instructions and data. Initially, these are loaded from the main memory into the L2 cache because they are not present in the cache. As the CPU accesses elements, they are brought into the L1 cache for faster access in subsequent iterations.
The GCC compiler translates high-level programming language code (e.g., C, C++) into machine code that can be executed by the CPU. It can optimise the code by reordering instructions, applying loop unrolling, and optimizing memory access patterns to enhance cache utilization and reduce cache misses.



# Padlet Questions

**L3 Basics Slide 13 Decoder:** Why does the decoder have to choose a unique word and what is this for?

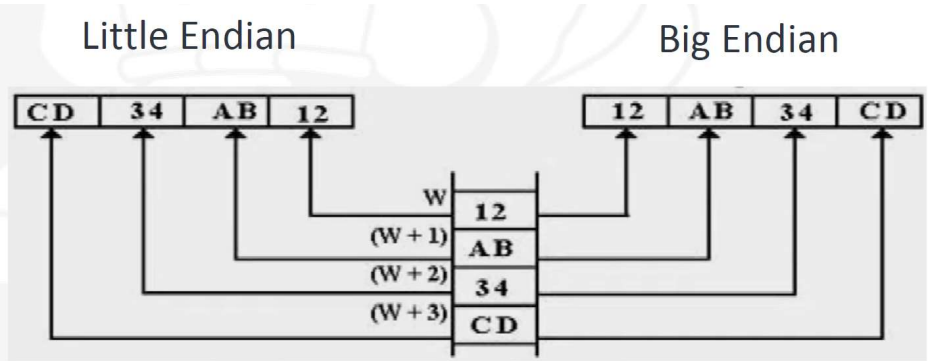



A memory address is a numeric value that identifies a specific location in computer memory where data is stored. Memory addresses are used by the CPU to access and manipulate data stored in the computer's memory subsystem. In the context of memory addressing, a "**word**" refers to the basic unit of data that a CPU can access from memory in a single operation. In processor-memory interaction, the decoder's role is crucial in selecting a unique word or location from memory.

# Padlet Questions

**L3 Basics Slide 14 Little Endian/Big Endian:** Is there any benefit to using one over the other and which is more commonly used?

Big endian and little endian are the two main ways to represent endianness. Endianess refers to the order in which bytes are stored in memory. The two terms came from Gulliver's Travels by Jonathan Swift. Two groups could not agree on at the little end or the big end an egg should be opened.
Just like the egg problem, there isn't necessarily a clear-cut benefit to using one over the other, as it often depends on the specific context and architecture of the system. However, there are some **considerations**:
**Memory Representation:** Little-endian representation is often considered more natural for processors because it aligns with the way numbers are written in decimal notation (least significant digits first). This can lead to slightly more efficient memory access in certain scenarios.



# Padlet Questions

**L3 Basics Slide 14 Little Endian/Big Endian:** Is there any benefit to using one over the other and which is more commonly used?

Big-endian representation is more commonly used in **network communication** protocols (like TCP/IP) due to its consistency across different systems. Network byte order is defined as big-endian, so conversions may be necessary when sending data between systems with different endianess.
Historically, both little-endian and big-endian architectures have been widely used in **processor architectures**. x86 and x86-64 architectures typically use little-endian, while many RISC architectures like SPARC and PowerPC used big-endian.
Endianess can impact **how data is aligned in memory**, which can affect memory access patterns and performance in some cases. This is more of a concern in low-level programming and systems optimization.

System designers and software developers. **Cross-platform communication** and **data serialisation**.

# Padlet Questions

**Exercise 1 Instruction Format:** in the Exercise 1, Q8 uses ADD, SUB and LW instructions. The format of these instructions is different to that used in the lab practicals. Do you have a document where you can tell us how we interpret each instruction, aka does ADD R3 R4 R2 mean add R3 and R4 and load result to R2 or does it mean add R4 and R2 and load result to R3?

We've clarified in the lecture and in the labs about it. Students are expected to be able to use various instruction set architectures. The instructions we use in the labs are **hypothetical** and are made artificially for the emulator and the labs; the instructions we use apart from the labs (e.g., in the lectures, exercises, and quizzes) are **MIPS** instruction set, which is a widely used ISA. You can find the MIPS instruction sheet and lab's Instruction Reference Sheet on VLE.

# SYS3 Exercises/Quizzes Discussions

# Discussion Forum

# Discussion Forum - Key Themes for Discussion

**1. Computer Organization Fundamentals**

Revisit foundational concepts in processor design, including architecture, components, and memory systems.

**2. Performance Evaluation**

Analyse metrics, trade-offs, and methodologies for assessing processor performance.

# Discussion Forum - Key Themes for Discussion

UNIVERSITY of York

**3. Modern Challenges & Trade-Offs**

Link classical concepts to contemporary issues like power efficiency and multi-core scaling.

TEF Gold Teaching Excellence Framework

# Discussion Forum - Computer Design & Evaluation

Pick one or more of the following questions, discuss with your small group and add comments to the Padlet discussion board:

**1. General-Purpose Register Computers**

What are the advantages and disadvantages of the three most common types of general-purpose register computers? Register-register, Register-memory, and Memory-memory.

**2. Interpreting Memory Addresses**

What are the issues with byte order in interpreting memory addresses?

**3. Alignment vs Misalignment**

Why would someone design a computer with alignment restrictions?

# Discussion Forum - Computer Design & Evaluation

**4. Parallelism**

Parallelism at multiple levels is now the driving force of computer design, with energy and cost being the primary constraints. Can you name two basic kinds of parallelism in applications? Examples?

**5. Data-Level and Task Level Parallelism**

What are the major ways computer hardware can exploit data-level and task level parallelism in applications?

**6. Benchmarks**

What is SPEC benchmark? Why do we need benchmarks?

**7. Performance Metrics**

Can you name any typical computer performance measuring metrics?

# Discussion Forum - Computer Design & Evaluation

**8. Memory Hierarchy**

Why is memory hierarchy critical to system performance?

**9. Amdahl's Law**

How does Amdahl's Law influence performance optimisation?

**10. Power Efficiency**

How do power constraints impact modern processor design?

# Discussion Forum - Approach

## 10 mins Break

## Small Group Discussions

– Form a small group with someone nearby, or feel free to move around and join your friends

– Pick one or more applications

– 15 mins group discussion

– Add comments/summary as a group to the Padlet discussion board

**https://padlet.com/pengchengliu83/week-2-discussion-forum-eo7kwvckdxevx8kq**

# Discussion Forum - Computer Architecture Basics

UNIVERSITY of York

**1. What are the advantages and disadvantages of the three most common types of general-purpose register computers? Register-register, Register-memory, and Memory-memory.**

| Type | Advantages | Disadvantages |
|---|---|---|
| Register-register (0, 3) | Simple, fixed-length instruction encoding. Simple code generation model. Instructions take similar numbers of clocks to execute (see Appendix C). | Higher instruction count than architectures with memory references in instructions. More instructions and lower instruction density lead to larger programs. |
| Register-memory (1, 2) | Data can be accessed without a separate load instruction first. Instruction format tends to be easy to encode and yields good density. | Operands are not equivalent since a source operand in a binary operation is destroyed. Encoding a register number and a memory address in each instruction may restrict the number of registers. Clocks per instruction vary by operand location. |
| Memory-memory (2, 2) or (3, 3) | Most compact. Doesn't waste registers for temporaries. | Large variation in instruction size, especially for three-operand instructions. In addition, large variation in work per instruction. Memory accesses create memory bottleneck. (Not used today.) |

The notation (m, n) means m memory operands and n total operands. In general, computers with fewer alternatives simplify the compiler's task since there are fewer decisions for the compiler to make. Computers with a wide variety of flexible instruction formats reduce the number of bits required to encode the program. The number of registers also affects the instruction size since you need log2 (number of registers) for each register specifier in an instruction. Thus, doubling the number of registers takes 3 extra bits for a register-register architecture, or about 10% of a 32-bit instruction.

TEF Gold Teaching Excellence Framework

# Discussion Forum - Computer Architecture Basics

UNIVERSITY of York

**2. What are the issues with byte order in interpreting memory addresses?**

As we discussed in the lecture, There are two different conventions for ordering the bytes within a larger object. Little Endian byte order puts the byte whose address is “x . . . x000” at the least-significant position in the double word (the little end).

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |

Big Endian byte order puts the byte whose address is “x . . . x000” at the most significant position in the double word (the big end).

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

When operating within one computer, the byte order is often **unnoticeable**—only programs that **access the same locations as both words and bytes**, can notice the difference. Byte order is a problem when **exchanging data among computers with different orderings**, however. Little Endian ordering also fails to match the normal ordering of words when strings are compared. Strings appear “SDRAWKCAB” (backwards) in the registers.

A second memory issue is that in many computers, accesses to objects larger than a byte must be aligned.

# Discussion Forum - Computer Architecture Basics

UNIVERSITY of York

**3. Why would someone design a computer with alignment restrictions?**

Misalignment causes **hardware complications**, since the memory is typically aligned on a multiple of a word or double-word boundary. A misaligned memory access may, therefore, **take multiple aligned memory references**. Thus, even in computers that allow misaligned access, programs with aligned accesses run faster.

Even if data are aligned, supporting byte, half-word, and word accesses requires an alignment network to align bytes, half words, and words in 64-bit registers.

# Discussion Forum - Principles of Computer Design

**4. Parallelism at multiple levels is now the driving force of computer design, with energy and cost being the primary constraints. Can you name two basic kinds of parallelism in applications?**

Data-Level Parallelism (DLP)

DLP occurs when **the same operation is performed on multiple data elements simultaneously**.

The same instruction is applied to many pieces of data.

Common in loops and array operations.

Exploited using vector processors, SIMD instructions, and GPUs.

# Discussion Forum - Principles of Computer Design

**4. Parallelism at multiple levels is now the driving force of computer design, with energy and cost being the primary constraints. Can you name two basic kinds of parallelism in applications?**

Data-Level Parallelism (DLP)

**Example:**

Adding two arrays:

```
for (i = 0; i < 100; i++)
    C[i] = A[i] + B[i];
```

All additions can be performed in parallel.

**Benefit:** Improves performance by processing large amounts of data at once.

# Discussion Forum - Principles of Computer Design

**4. Parallelism at multiple levels is now the driving force of computer design, with energy and cost being the primary constraints. Can you name two basic kinds of parallelism in applications?**

**Task-Level Parallelism (TLP)**

Task-level parallelism occurs when **different tasks or threads execute simultaneously**.

Different instructions operate on different data.

Common in multi-threaded and multi-core systems.

Each core executes a separate task independently.

# Discussion Forum - Principles of Computer Design

**4. Parallelism at multiple levels is now the driving force of computer design, with energy and cost being the primary constraints. Can you name two basic kinds of parallelism in applications?**

**Example:**

One thread handles user input

Another thread performs calculations

Another thread saves data to disk

**Benefit:** Improves overall system throughput and multitasking.

# Discussion Forum - Principles of Computer Design

**5. What are the major ways computer hardware can exploit data-level and task level parallelism in applications?**

**Instruction-Level** Parallelism exploits data-level parallelism at modest levels with compiler help using ideas like pipelining and at medium levels using ideas like speculative execution.

**Vector** Architectures and Graphic Processor Units (GPUs) exploit data-level parallelism by applying a single instruction to a collection of data in parallel.

**Thread-Level** Parallelism exploits either data-level parallelism or task-level parallelism in a tightly coupled hardware model that allows for interaction among parallel threads.

**Request-Level** Parallelism exploits parallelism among largely decoupled tasks specified by the programmer or the operating system.

# Discussion Forum - Principles of Computer Design

**6. What is SPEC benchmark?**

SPEC, which stands for the Standard Performance Evaluation Corporation, is a non-profit organisation that develops and maintains standardized benchmarks for evaluating the performance of computer systems.

SPEC benchmarks are widely used in the computer industry to assess the performance of various hardware and software components, including processors, compilers, and operating systems. The goal of SPEC is to provide a fair and unbiased platform for comparing different systems and architectures.

SPEC benchmarks cover a range of computing domains, and each benchmark is designed to measure specific aspects of system performance. Some of the well-known SPEC benchmarks include: SPEC CPU (SPECint and SPECfp), SPECjvm (Java virtual machines), SPECweb (web servers), SPECpower (power efficiency of computer systems), SPECviewperf (graphics performance), etc.

# Discussion Forum - Principles of Computer Design

**6. Why do we need benchmarks?**

It provides a standardized way to **evaluate the performance** of hardware components, software applications, and overall systems.

It creates a level playing field for comparing different products, systems, or architectures. Standardized tests ensure that the **comparison is fair**, **unbiased**, and based on a common set of criteria.

Manufacturers and developers use benchmarks during the **design and development** process to **optimise** their products through identifying performance bottlenecks and areas for improvement.

# Discussion Forum - Principles of Computer Design

**6. Why do we need benchmarks?**

Researchers use them to evaluate the performance of new algorithms, architectures, or technologies for **Research and Innovation.** Advance the state of the art and push the boundaries of what is possible in computing.

It assists organizations in making **cost-effective decisions** when selecting hardware or software solutions.

It is used to assess the **compatibility of software applications** with different hardware configurations.

It contributes to the **standardisation of performance measurement methodologies.** This standardisation ensures consistency in evaluations across the industry, making it easier for consumers and professionals to understand and interpret benchmark results.

# Discussion Forum - Principles of Computer Design

**7. Can you name any typical computer performance measuring metrics?**

Clock speed **(Frequency)** is measured in Hertz (Hz) and represents the number of clock cycles a processor can execute per second. A higher clock speed generally indicates faster processing.

Instructions Per Cycle **(IPC)** measures the average number of instructions a processor can execute in a single clock cycle. It reflects the efficiency of the processor's architecture.

Cycles Per Instruction **(CPI)** is the inverse of IPC, indicating the average number of clock cycles needed to execute one instruction. Lower CPI values suggest better efficiency.

**Throughput** measures the amount of data that can be processed within a specific time period. It is often used to assess the performance of input/output operations, storage devices, and network connections.

# Discussion Forum - Principles of Computer Design

**7. Can you name any typical computer performance measuring metrics?**

Response Time **(Latency)** is the time it takes for a system to respond to a given request. It's crucial for user experience and is often measured in milliseconds.

**Bandwidth** measures the maximum rate of data transfer across a network or communication channel. Higher bandwidth values indicate faster data transfer capabilities.

**Cache Hit Rate** measures the percentage of memory accesses that result in a successful cache hit. A higher hit rate indicates more efficient use of cache memory.

**Memory throughput** measures the rate at which data can be read or written to memory, while **memory latency** measures the time it takes for the processor to access data from memory.

# Discussion Forum - Principles of Computer Design

**7. Can you name any typical computer performance measuring metrics?**

**Disk I/O Performance** metrics include measures such as read and write speeds, response times, and the number of I/O operations per second for storage devices.

For graphics-intensive tasks, metrics such as frames per second (FPS) and rendering times are essential for assessing the **performance of graphics** cards and GPUs.

**Energy Consumption** metrics measure the amount of power a system consumes during operation. **Power efficiency** is crucial, especially in mobile devices and data centers.

**Fault Tolerance and Reliability** metrics assess a system's ability to continue operating in the presence of failures and its overall reliability.

**Benchmark Scores** from standardized benchmarks (e.g., SPEC benchmarks) provide an overall measure of a system's performance across different workloads.

# Discussion Forum - Computer Organization Fundamentals

**8. Memory Hierarchy : Why is memory hierarchy critical to system performance?**

Memory hierarchy plays a critical role in processor performance because it helps **bridge the speed gap** between the CPU and memory, ensuring efficient data access and minimising delays. Without an optimised memory hierarchy, processors would spend excessive time waiting for data, leading to performance bottlenecks.

The Speed Gap Between CPU and Memory. Modern processors operate at GHz speeds, executing billions of instructions per second. However, memory access (especially to DRAM) is significantly slower compared to CPU execution speeds. This disparity creates a memory bottleneck where the CPU is forced to stall while waiting for data.

A well-structured memory hierarchy **reduces latency** by ensuring that frequently accessed data is stored closer to the processor.

# Discussion Forum - Computer Organization Fundamentals

UNIVERSITY of York

**8. Memory Hierarchy : Why is memory hierarchy critical to system performance?**

Structure of the Memory Hierarchy. The memory hierarchy is designed to provide a **trade-off between speed, size, and cost**:

| Level | Type | Access Speed | Size | Cost per Bit |
|---|---|---|---|---|
| Registers | Inside CPU | Fastest (~1 cycle) | KBs | Very High |
| L1 Cache | On-chip Cache | 1-4 cycles | 32-128 KB | High |
| L2 Cache | On-chip Cache | 10-20 cycles | 256KB-8MB | Medium |
| L3 Cache | Shared Cache | 40-50 cycles | 4-64MB | Medium-Low |
| RAM (DRAM) | Main Memory | 100-200 cycles | GBs | Low |
| SSD / HDD | Storage | 100,000+ cycles | TBs | Very Low |

Each level provides a **trade-off**: Faster memory is smaller and more expensive. Slower memory is larger and cheaper.

By **caching frequently accessed data** in faster memory levels, the CPU can minimize time wasted waiting for data.

# Discussion Forum - Computer Organization Fundamentals

**8. Memory Hierarchy : Why is memory hierarchy critical to system performance?**

Impact of Memory Hierarchy on Performance. Cache Effectiveness – The Principle of **Locality**. Temporal Locality: If a memory location is accessed, it is likely to be accessed again soon. Spatial Locality: If a memory location is accessed, nearby locations are likely to be accessed soon.

**Reducing Memory Latency**. A cache hit occurs when the needed data is found in the cache, leading to fast access. A cache miss forces the CPU to fetch data from lower levels, significantly increasing access time.

Impact on **Multi-Core Processors**. Multi-core CPUs use a shared L3 cache, which helps coordinate data access. Cache coherence protocols (e.g., MESI – Modified, Exclusive, Shared, Invalid) prevent inconsistencies when multiple cores access the same data.

# Discussion Forum - Computer Organization Fundamentals

**8. Memory Hierarchy : Why is memory hierarchy critical to system performance?**

Power Efficiency. **Power consumption** is a key design factor:

DRAM consumes more power than caches, and excessive memory access increases energy use.

CPU stalls due to cache misses lead to wasted power.

Techniques like clock gating, DVFS (Dynamic Voltage and Frequency Scaling), and scratchpad memories optimize energy usage.

# Discussion Forum - Performance Evaluation

**9. Amdahl's Law**

How does Amdahl's Law influence performance optimisation?

$$S = \frac{1}{(1-P) + \frac{P}{N}}$$

Where:

**S** = Overall speedup

**P** = Proportion of the program that can be parallelised

**1 - P** = Proportion of the program that remains sequential

**N** = Number of processors (or parallel execution units)

This formula shows that even if we infinitely increase **N** (processors), the speedup is ultimately limited by the **sequential portion (1 - P)** of the program.

# Discussion Forum - Performance Evaluation

UNIVERSITY of York

**9. Amdahl's Law**

How does Amdahl's Law influence performance optimisation?

$$S = \frac{1}{(1-P) + \frac{P}{N}}$$

Practical Interpretation

If 90% of a program can be parallelised (P = 0.9), but 10% is inherently sequential (1 - P = 0.1), even with infinite processors, the maximum speedup is only 10×.

If only 50% of a program can be parallelised, the best speedup is just 2×, no matter how many cores we add.

This means that focusing solely on increasing the number of processors may not always yield significant performance gains.

# Discussion Forum - Performance Evaluation

**9. Amdahl's Law**

How does Amdahl's Law influence performance optimisation?

$$S = \frac{1}{(1-P) + \frac{P}{N}}$$

Key Implications on Performance Optimisation

Diminishing Returns of Parallelism: Adding more cores or execution units has a limited effect once the sequential portion becomes the bottleneck.

Example: If a program has a 15% sequential portion, even using 100 processors results in a speedup of only 5.9× instead of 100×.

# Discussion Forum - Performance Evaluation

**9. Amdahl's Law**

How does Amdahl's Law influence performance optimisation?

$$S = \frac{1}{(1-P) + \frac{P}{N}}$$

Key Implications on Performance Optimisation

Importance of Reducing the Sequential Portion: Instead of just increasing parallel execution, optimising the sequential part of a program can have a greater impact.

Methods include: Algorithm optimisation (e.g., replacing inefficient loops with faster computations); Reducing memory latency (e.g., better caching techniques)Using hardware accelerators (e.g., GPUs, FPGAs, AI accelerators).

# Discussion Forum - Performance Evaluation

**9. Amdahl's Law**

How does Amdahl's Law influence performance optimisation?

$$S = \frac{1}{(1-P) + \frac{P}{N}}$$

Key Implications on Performance Optimisation

Trade-off Between Parallelisation and Overhead: Some parallelisation techniques introduce overhead (e.g., thread synchronisation, inter-core communication). If the overhead exceeds the gains from parallelism, it can reduce or even negate performance improvements.

# Discussion Forum - Performance Evaluation

**9. Amdahl's Law**

How does Amdahl's Law influence performance optimisation?

$$S = \frac{1}{(1-P) + \frac{P}{N}}$$

Key Implications on Performance Optimisation

Multi-Core & Distributed Systems Design: Amdahl's Law guides CPU and GPU architecture:CPUs have strong single-threaded performance for handling sequential tasks efficiently. GPUs focus on massive parallelism, but Amdahl's Law explains why not all workloads benefit equally.

In distributed computing (e.g., cloud computing, HPC), performance bottlenecks due to network latency and data consistency further limit scalability.

# Discussion Forum - Performance Evaluation

**10. Power Efficiency**

How do power constraints impact modern processor design?

In modern processor design, higher performance often leads to increased power consumption and heat generation.

Key power constraints:

- **Power density and thermal dissipation**. Power density refers to the amount of power consumed per unit area of the processor. Excessive power density leads to thermal hotspots, which can degrade performance and damage hardware.
- **Battery life** in mobile devices. Smartphones, tablets, and laptops rely on battery power, making low-power design essential.
- **Data centre power consumption**. Servers and data centers run high-power CPUs and GPUs 24/7. Cooling costs and environmental impact.

# Discussion Forum - Performance Evaluation

**10. Power Efficiency**

How do power constraints impact modern processor design?

Architectural Techniques to Address Power Constraints

- **Multi-core and heterogeneous architectures**. Instead of increasing clock speed, processors use multiple cores to distribute workload efficiently. Heterogeneous computing combines high-power cores (for demanding tasks) and low-power cores (for background processes).

- **Lowering operating voltage**. Since power consumption scales with the square of voltage, lowering voltage significantly reduces power usage. Using adaptive voltage scaling (AVS) dynamically adjusts voltage levels.

- **Specialised accelerators**. General-purpose CPUs are power-hungry for certain tasks. AI chips (TPUs, NPUs), FPGAs, and ASICs consume less power for specific workloads.

# Discussion Forum - Performance Evaluation

**10. Power Efficiency**

How do power constraints impact modern processor design?

**Thermal Management** Techniques

- Excess power consumption results in heat generation, requiring effective cooling strategies.
- Common solutions: Efficient heat sinks & liquid cooling (for desktops & servers); Thermal throttling (reduces clock speed to prevent overheating); Energy-efficient software optimization (e.g., reducing unnecessary background processes).

# Closing Remarks and Announcement

- **Next Week (Week 3)**:

**Lecture:** Next week, our discussion will be focused on some principles of the RISC Pipeline. Also, we will be having our first Tutorial Lecture on Performance.

**Lab:** In Weeks 3 and 4 we will be exploring the idea of structural hazards in a CPU pipeline, and possible solutions. You will start with some basic ideas, then develop an algorithm to solve structural hazards in the pipeline. In Week4 an addittional help sheet will be released to assist those who have not been able to devise a suitable idea to implement. Solutions to algorithm problems will be released at a later date.

**Exercises/Quizzes:** Quiz 1 will be release on 20/02/2026 (Week 2 Friday). This is a formative quiz based on week 1, week 2 and week 3 lectures, responses will be automarked. You can try as many times as you like till you get a perfect score. Quiz 1 will be discussed in the Lecture Review session on 12/03/2026 (Week 5 Thursday).

**Week 3 Lecture Review**

**Guest Lecture 1**

**Speaker: Dr Wei Wu**

**Title:** RISC-V: A Brief History and Roadmap to 2036

**Abstract:** As a young and open ISA standard, the RISC-V has the potential to capture a quarter of the global processor market share within its 20-year lifespan. Some bold predictions even suggest that RISC-V will become the most dominative ISA in the worldwide IT industry by 2036, serving as a cornerstone of the global open-source ecosystem due to its open and free nature.

In this lecture, the speaker will begin with the historical context of RISC-V's emergence, explaining the historical opportunities and inevitability of its creation as an open and free ISA. Furthermore, the speaker will showcase the remarkable achievements RISC-V has made in the software and open-source chip ecosystems, particularly the policy support from EU, China, Brazil, and India.

## Week 3 Lecture Review

## Guest Lecture 1

## Speaker: Dr Wei Wu

**Title:** RISC-V: A Brief History and Roadmap to 2036

**Biography:** Mr. Wei Wu is the founder and director of Project Jiachen, whose mission is to make RISC-V the most dominant instruction set architecture for all IT scenarios before 2036, achieving commercial prosperity. Currently, more than 50 companies, research institutions, and developers worldwide have joined the project, covering almost all major RISC-V IP companies. To achieve this vision, he founded PLCT Lab in 2019 and serves as its project director. Improving the RISC-V software ecosystem has been set as one of PLCT lab's most important strategic objectives. His team maintains a series of key open-source infrastructures, including OpenJDK RISC-V backend, V8 RISC-V backend, Spidermonkey RISC-V JIT backend, LuaJIT RISC-V downstream implementation, as well as GNU Toolchain and Clang/LLVM RISC-V backend. The TARSIER project, which he launched at the end of 2021, has taken a significant step forward in RISC-V desktop computing and Linux distribution support. In October 2025, he joined KUBUDS, a company he co-founded in 2023, to help RISC-V companies achieve greater success in the commercial industrial software ecosystem.

Wei has served as a director of the LLVM Foundation Board of Directors (BoD) since 2022. He is very passionate about educating compiler techniques and RISC-V to engineers and amateurs. He often gives talks and tutorials among amateurs and college students in local meetups. He also collaborates with college teachers, providing RISC-V software lectures and experiment courses.

# Thanks

# See You Next Week !